Weld Release Study: Task List & Data Collection Plan (Draft v0.3 — Solo, Mocked GitHub)

Study format. Public release with a guided ~35–45 minute task set, student-focused but not student-limited. Single-participant (solo) throughout — no live teammates, no group scheduling. A pre-seeded sample project provides a frozen "team" context. **All GitHub content is seeded/mocked inside Weld — no live GitHub API is used during the study, and write controls are hidden behind a `STUDY_MODE` flag.** An optional trigger-based bot adds limited live reactions at one or two key moments, but the study is fully valid on static seed content alone. IRB approval secured before data collection begins. Target timeline: ~1 month (extendable if needed).

Core design principle. The bot is an *enhancement, never a dependency.* Every task must be completable and every measure must be collectable if the bot is disabled or fails. A bot glitch should degrade one participant's realism slightly — never invalidate their data.

Why mocked GitHub. The live GitHub integration supports writes (create/comment/close), which means a single shared repo would be corrupted by the first participant and could not present an identical seed state to the next. Since every GitHub touchpoint in the study tasks is *view* or *link* only — never a write — there is no need for a live repo. Seeding GitHub content as static fixture data inside Weld gives every participant a pixel-identical, always-available view with zero API dependency (no rate limits, network errors, or state drift), and makes corruption physically impossible. **Limitation to note in the paper:** participants interacted with representative repository content rather than a live repository — fully defensible given view/link-only tasks.

Critical week-1 check. Confirm whether Weld's GitHub view can render from seeded local data or is hardwired to the live API. If hardwired, either (a) adapt it to read from seed data, or (b) fall back to a real repo connected with a *read-only token* plus graceful error handling. This is on the critical path for the pilot.

---

1. Study Structure at a Glance

Phase	Duration	Format	What it measures
0. Consent + screening	3–5 min	Web form before app access	Eligibility, demographics

Phase	Duration	Format	What it measures
1. Solo onboarding	10–12 min	Scripted	Feature discoverability, learnability, time-to-first-action
2. Workflow tasks	15–18 min	Lightly guided, solo	Cross-module integration use, realistic workflow patterns
3. Open exploration	5 min	Unguided	What people do when given freedom
4. Exit survey	5–8 min	Web form	NASA-TLX, SUS, qualitative feedback

Total: ~38–48 minutes. Target a median around 40 min in pilot testing; if it runs long, cut Task 2.3 first.

No scheduling required. Because the study is solo, participants complete it whenever they want via a self-serve link. This dramatically simplifies recruitment and removes no-show risk entirely.

2. The Pre-Seeded Sample Project

Every participant logs into the same pre-seeded fake project on first launch. This is the most important design decision in the study. The "team" is frozen history — realistic chat, cards, and code that demonstrate the integration story without requiring anyone to be live.

Project name. "PetTracker" — a fictional mobile app for tracking pet vet visits, vaccinations, and feeding schedules. Concrete enough to feel real, simple enough that no participant needs domain knowledge.

What's pre-seeded:

- **Board.** 3 columns (Backlog, In Progress, Done). ~12 task cards with realistic titles, descriptions, labels, priorities. 3 cards link to GitHub issues; 2 link to PRs.
- **Chat.** A #general and a #pettracker channel. ~30 seeded messages spanning the prior "week" — casual updates, questions about specific cards, PR links, and one multi-message decision thread the participant will need to find and read.

- **Fake teammates.** 4 teammates (e.g., Sarah, Alex, Priya, Marcus) with avatars and a believable activity history. By default they are *inactive* during the session — they exist as authors of the seed content only. The optional bot (see §7) can voice one of them at scripted moments.
- **GitHub content (mocked/seeded — no live API).** A small set of fixture data rendered in Weld's GitHub view exactly as the live integration would display it: realistic commit history, 3 open issues, 2 open PRs, a README. Seeded into Weld's own database alongside the cards and chat. Write controls (create issue/PR, comment, close) are hidden via the `STUDY_MODE` flag so participants — including during open exploration — have no write affordances and cannot corrupt the shared state. No GitHub account or repo access is required of participants.

Why the frozen team works for a solo study. The integration signal you care about — does the participant follow chat→card links, do they re-read context that's already cross-referenced, do they navigate between board and GitHub — is fully present in static content. A participant reconstructing and acting on existing team context tests integration *better* in some ways than a live discussion, because the information they need is already sitting in the cross-references.

3. Task List (Solo)

Phase 1 — Onboarding (10–12 min)

Scripted, identical across all participants. Ensures everyone touches every core feature before the workflow phase, and produces comparable time-to-complete metrics.

Task 1.1 — Find your bearings (2 min).

> "You've just been added to the PetTracker team. Take a look around. When ready, find the card titled 'Add vaccination reminders' on the board, open it, read the description, then close it."

Measures. Time to locate a specific card. Orientation behavior — search vs. scroll vs. filter.

Task 1.2 — Make your mark (2 min).

> "Create a new task card in the Backlog column titled 'Set up unit tests for feeding tracker' with the label 'testing' and assign it to yourself."

Measures. Card creation flow. Time to first successful create. Label/assignment discovery.

Task 1.3 — Connect the dots (3 min). *(critical task)*

> "Sarah's chat message from yesterday in #pettracker references a bug. Find the message, open the related task card directly from the chat link, and move that card to 'In Progress'."

Measures. The first moment integration matters: chat → board cross-reference. Whether users discover cross-references without being told they exist. If users struggle here, the seed needs more obvious linking cues.

Task 1.4 — Drop a note (2 min).

> "Post a message in #pettracker saying you've picked up the bug, and use an @ mention to tag Sarah."

Measures. Chat input flow, @ mention discovery, time-to-first-message. **Optional bot trigger point #1** — see §7.

Task 1.5 — Wrap up onboarding (1 min).

> "Open the GitHub view and find the open PR titled 'Refactor feeding schedule API'. Just locate it — no action needed."

Measures. GitHub module discovery and prominence in the UI.

Phase 2 — Workflow Tasks (15–18 min)

Redesigned for solo completion. Teammates are referenced as context, never required to respond. The tasks center on *reconstructing and acting on existing team context* — which is exactly the integration story.

Task 2.1 — Triage (5 min). *(highest-value task)*

> "Three new issues have been reported, shown as cards in the Backlog with the 'new-issue' label. Read each one, decide which is highest priority based on the descriptions and the recent discussion in #pettracker, set priorities on all three cards, and post a short message in #pettracker explaining your prioritization."

Measures. Cross-tool reasoning. Does the participant consult chat before prioritizing? Do they reference cards by ID/name/link in their explanation message? The priority assignments + the explanation message together reveal whether integrated context changes how they triage. Fully solo — no teammate response needed.

Task 2.2 — Take over a card (5 min).

> "Your teammate Sarah is going on leave. Take over her card 'Add vaccination reminders.' Read her status notes in the linked chat thread, then update the card's description so the team knows exactly where things stand and what's left to do."

Measures. Whether the participant follows the chat→card link, how much linked context they actually read before acting, and whether they re-state context that's already visible (a sign integration *isn't* reducing redundant effort) or build on it. **Optional bot trigger point #2** — a brief acknowledgment from "Marcus" can fire here if the bot exists, but the task completes fine without it.

Task 2.3 — Record a decision (5 min, cut first if running long).

> "The team has been debating PostgreSQL vs. SQLite for the local cache in a #pettracker thread started by Alex. Read the thread, then create a new card capturing the decision the team reached and its rationale, with appropriate labels. If your tool lets you reference the original discussion from the card, do so."

Measures. Finding and synthesizing cross-referenced information. Whether the resulting card actually links back to the chat thread, or whether the participant recreates context manually. This directly tests whether integration reduces context-reconstruction cost.

Phase 3 — Open Exploration (5 min)

> "You have 5 minutes to explore Weld however you like. No specific task — try features you haven't used, customize your view, or just poke around."

Measures. Unguided feature gravitation. Heatmap of what people try when free. Lowest rigor, often highest insight about what users find compelling.

GitHub during exploration. With write controls hidden via `STUDY_MODE`, participants who open the GitHub view simply see the read-only seeded content — realistic and harmless. Watching whether participants *try* to find write actions (and don't) is itself mild signal. The `STUDY_MODE` hide ensures no broken-looking error appears.

Phase 4 — Exit Survey (5–8 min)

See §5.

4. Telemetry / Behavioral Data to Collect

Every task maps to at least one quantitative measure; every measure answers a specific question. All captured automatically by Weld's analytics layer.

4.1 Session-level metadata

Anonymized participant ID; session start/end timestamps; phase transition timestamps; browser/OS/screen resolution; completion vs. abandonment and the point of abandonment.

4.2 Per-task timing

For each task: presented timestamp; completed timestamp (via "I'm done" button); completed vs. skipped; number of times the instruction popup was re-opened (a confusion proxy).

4.3 Feature-level events

Board. card_created, card_opened, card_edited, card_moved (from/to column), card_assigned, card_labeled, card_priority_set, card_deleted, card_searched (query), board_filtered (filter type).

Chat. message_sent (channel, length, contains_mention, contains_link), message_edited, message_deleted, channel_switched, thread_opened, mention_clicked, link_clicked (target type: card / PR / issue / external).

Cross-reference events *(most important — instrument these specifically if not already present).*

- card_referenced_from_chat
- chat_referenced_from_card
- pr_referenced_from_card
- card_referenced_from_pr

GitHub. github_view_opened, pr_opened, issue_opened, pr_linked_to_card, issue_linked_to_card.

Navigation. view_changed (from_view, to_view), search_used (query, results_clicked_index).

4.4 Computed metrics (derived at analysis time)

- Time-to-first-action per phase
- **View-switch rate** (switches/min between board/chat/GitHub) — *key context-switching metric*
- **Cross-reference rate** (% of card opens arriving via a chat/GitHub link vs. direct navigation)
- Task completion accuracy (scored against rubric, §6)
- **Context-reconstruction effort** (in Tasks 2.2/2.3, whether participant re-states already-linked info — coded from card text + messages)

4.5 What NOT to collect

State this explicitly in IRB + consent — it eases the exempt/expedited determination:

- No keystrokes (final message text only)
- No mouse-movement or hover tracking
- No screen/camera recording
- No data from other browser tabs
- No data about projects other than PetTracker

5. Exit Survey (5–8 min)

All Likert items 7-point unless noted.

5.1 NASA-TLX (6 items). Mental, Physical, Temporal Demand; Performance; Effort; Frustration. Raw/un-weighted version.

5.2 System Usability Scale / SUS (10 items). Standard, comparable to published norms.

5.3 Integration-specific custom items (4 items):

> "When I needed to find information across different parts of Weld (board, chat, GitHub), it was easy." (1–7) "I noticed moments where features in Weld were connected to each other in helpful ways." (1–7) "Compared to using separate apps for tasks, chat, and code, working in Weld felt:" (1 = much harder … 7 = much easier) "I would consider using Weld for a real class or personal project." (1–7)

5.4 Open-ended (3 items):

> "What was the most useful moment of working in Weld today?" "What was the most frustrating moment?" "If you could change one thing, what would it be?"

5.5 Demographics. Age range, year in school (if applicable), primary languages, years of programming experience, prior use of Linear/Jira/GitHub Projects/Slack/Discord. Collect only what you'll analyze.

6. Manual Coding Rubric (Post-Hoc)

Two researchers independently code per participant:

- Task completion score per task (0 = not attempted, 1 = attempted/incorrect, 2 = partial, 3 = correct)

- Whether chat context was consulted before acting in Tasks 2.1–2.3 (yes/no, from event stream)
- Context-reconstruction behavior in Tasks 2.2/2.3 (built-on-existing vs. manually-recreated)
- Notable confusion moments (qualitative flags)

Resolve disagreements by discussion; report Cohen's kappa on a 20% sample.

7. The Optional Bot — Scope and Spec

Status: enhancement, build only if Phase 1–2 are working on static seed first.

Purpose. Add a touch of liveness at one or two moments so the workspace doesn't feel completely frozen, without making any task depend on a response.

Trigger model. Trigger-based (responds to a specific participant action), not timer-based or keyword-parsing. Each trigger has a single pre-written response and a short delay (15–30s) to feel human.

Recommended triggers (pick one or two, not more):

1. **After Task 1.4** (participant @-mentions Sarah saying they picked up the bug): bot, voicing Sarah, replies after ~20s with a fixed message like *"Thanks for grabbing that! I left some notes on the card."* Low risk — the participant just posted, so a reply is natural and the content is generic enough to fit almost any phrasing they used.

2. **During Task 2.2** (participant updates the handoff card): bot, voicing Marcus, posts ~25s later *"Appreciate you picking this up while Sarah's out 👍"*. Again generic, reaction-agnostic.

Hard design rules for the bot:

- One canned response per trigger. No branching, no NLP, no attempt to "understand" the message.
- If the participant never fires the trigger (e.g., skips the @ mention), the bot simply stays silent. Nothing breaks.
- Bot messages are clearly logged with a `bot_message` flag so they're excluded from participant behavioral analysis.
- The bot must be disable-able with a single config flag, and the study runs identically with it off. Pilot both ways.

What the bot must NOT do. No live conversation, no answering participant questions, no reacting to anything outside the two scripted triggers. The moment a participant gets a canned

non-sequitur, the illusion breaks and that data becomes suspect — so keep the surface area tiny.

Build estimate. A two-trigger, single-response-each bot is roughly a day of work on top of an existing chat backend (listen for an event, wait N seconds, post a fixed message as a given user). If it's taking longer than that, ship Version A (static only) and move on.

8. Open Implementation Questions

1. **Hosting/access.** Recommend invitation-only via a study landing page (consent → screening → app access) during the study; open signup afterward. Cleaner data, simpler IRB.
2. **Compensation.** Even $10–15 (gift card) meaningfully lifts completion and survey quality. Confirm with professor what departmental funds allow.
3. **Pilot before launch.** Run 2 full sessions (with and without the bot) before opening to real participants. Most issues surface in the first pilot.
4. **Abandonment handling.** Since it's self-serve, some participants will quit partway. Log where they stopped; decide a minimum-completion threshold (e.g., finished through Phase 2) for inclusion in primary analysis.

9. Sample Size

Solo + self-serve makes recruitment far easier than the group design. Targets:

- **40–60 participants:** solid for CHI Late-Breaking Work, CSCW Posters, ICSE NIER, VL/HCC.
- **80+:** competitive for full CHI/CSCW.
- **<30:** preliminary/workshop territory.

Self-serve solo studies can realistically hit 60+ in 3–4 weeks of active recruitment through course lists and dev communities.

10. What This Plan Buys You

- "Participants completed cross-module tasks (e.g., Task 1.3, board↔chat) X seconds faster than within-module tasks of comparable complexity."
- "Y% of card opens came from chat/GitHub references rather than direct navigation, suggesting cross-references are a primary discovery path."
- "Self-reported integration-helpfulness correlated with [or diverged from] objective cross-reference usage" — preserves the perception–behavior gap angle.
- A qualitative section grounded in open-ended survey responses.

Path to a longer paper: expand N, add a longitudinal follow-up ("did anyone keep using Weld a month later?"), or add a between-subjects baseline condition (Weld vs. separate-tools trio).

11. Timeline (~1 month, extendable)

This is compressed to fit a one-month target. The hardest constraint is IRB approval — start it day one. If IRB slips, the collection window (weeks 3–4) is what extends.

Week	**Activity**
1	Submit IRB immediately. Confirm GitHub view can render from seed data (critical check). Instrument telemetry (esp. cross-reference events). Build `STUDY_MODE` flag. Draft seed content (cards, chat, GitHub fixtures).
2	Finish seed content. Build study landing page + consent flow + exit survey. Run 1–2 internal pilots. Iterate on task wording and timing. (Optional bot only if everything else is done.)
3	IRB approval expected. Open recruitment — push self-serve link to course lists + dev communities. Begin collection. Monitor telemetry pipeline live.
4	Continue collection to target N. Clean data as it arrives. Begin post-hoc coding and analysis.
5+ (if extended)	Top up sample size, finish coding, draft paper.

If you only get one month total: prioritize hitting ~40 completed participants and a clean dataset over polish. The bot, longitudinal follow-up, and any baseline condition are all cuttable. A clean 40-participant solo study with good telemetry is a publishable short paper; a half-finished ambitious one is not.

---

v0.3 — solo + mocked-GitHub redesign, one-month timeline. Next deliverables once locked: IRB application materials, consent form, in-app welcome/tutorial flow, seed project content (cards, chat messages, GitHub fixture data), exit survey form, and (optional) bot trigger spec.

May 28, 2026

1. RQ1: charaterizing the user behaviors
2. RQ2: how effective is AI agent usage in Weld?

- Data collection, logging events
- AI agent.